~~Uninformed Search:~~

State representations:

1) Atomic → we don't know the internal structure and can't understand what is the diffeence in 2 states internally
2) Propositional / factored → state consists of state variables that has values and we can identify what changed internally
3) Relational → state is described based on relations of the internal stru
4) First-order → relational with functions of objects

Search strategy → picking the order of node expansion

evaluting based on:

1) completeness - find soln if exists
2) time complexity
3) Space complexity
4) optimality - always find least cost soln
5) systematicity - each state at most once

time and space complexity measured in terms of:

b - maximum branching factor of the search tree
d - depth of least cost soln
m - maximum depth of state space

Uninformed search strategies - use only information available in problem definition.

Uninformed search
↳ BFS
↳ DFS
↳ Depth-limited search
↳ Iterative deeping search

② BFS: shortest first
- complete (b is finite)
- time complexity: $O(b^d)$
- space complexity: $O(b^d)$
  ↳ maintain full frontier
- Optimal if all step cost is same
  ↳ queue of nodes to visit

BFS or DFS → BFS better in time and DFS in space
Memory is reusable while time is not
↳ but not for a single algo

BFS for depth 10 → memory 10 terabytes, 3 hours (not possible) ← space
for difficulty in navigating vast search spaces:
1) Beam search: maintain constant sized frontiers
④ When frontier becomes large prune worst nodes
- Not optimal and complete
if we add some notion of what is good node and what is bad not, it can be better (chat bots)

2) Iterative Deepening DFS:
⑤ → DFS but stop in the middle ⇒ Depth-limited search
→ for depth 0 to ∞:
  result ← depth-limited-search
  If result ≠ cutoff then return result
- ~~Not~~ complete
- Time complexity: $(d+1)b^0 + db^1 + (d-1)b^2 + \dots + b^d = O(b^d)$
  (no. of times root node is touched; d+1 layer)

① DFS:
- not complete depth can be ∞
- time complexity: $O(b^m)$
- space complexity: $O(bm)$
  ↳ just store path
  ↳ stack of nodes visited

negative costs
③ Uniform cost search: cheapest first
- complete (b is finite)
- time complexity: $O(b^{(C^*/\varepsilon)})$
- space complexity: $O(b^{(C^*/\varepsilon)})$
  C^* - lowest cost, c^* - optimal cost
- optimal
  ↳ queue of nodes to visit

- space complexity: $O(bd)$
- Optimal if step cost is same
  ↳ modified to explore uniform cost tree (iterative lengthening)
- Not systematic

⑥ Bidirectional search: search both ways from start state to goal state and from goal state to start state.
- branching factor: average branching factor going out of the node and into the node – fan out and fan in branching factor.
→ if fan out is lower – forward search
→ if fan in is lower – backward search
- when we are given the start state and goal test, we can't do both bidirectional search ↳ don't have a specific goal state but many goals
→ BFS in both direction
- complete* and optimal if uniform cost search in both directions
- time complexity: $O(b^{d/2})$
- space complexity: $O(b^{d/2})$

Repeated expansions are bigger issue for DFS than for BFS or IDDFS trying to remember all visited nodes
↳ space - exponential ↳ duplicate checking - expensive
⇒ Partial solution:
  ↳ checking if any children of a node have same state as parent
  ↳ checking if any children of a node have same state as any of ancestor of n.

Uninformed search - slow (blind)
↳ add heuristic estimate ⇒ informed search

Informed search (Heuristic search)
↳ difference b/w this & blind search (we are giving some more information)

- Node is selected for expansion based on an evaluation function that estimates cost to goal.

General tree search paradigm — fringe = frontier same as open list

```
fn tree search (root-node)
  fringe ← successor (root-node)
  while fringe not empty:
    node ← remove-first (fringe) ⇒ lowest f-value
    state ← state (node)
    if goal-test (state) return soln(node)
    fringe ← insert all (successor(node), fringe)
  return failure
```

General graph search paradigm — use closed list here

```
fn tree-search (root node)
  fringe ← successor (root-node)
  explored ← empty ⟵ new parameter
  while fringe not empty:
    node ← remove-first (fringe)
    state ← state (node)
    if goal-test (state) return soln (node)
    explored ← insert (node, explored)
    fring ← insert all (successor (node, fring, if node not in explored)
  return failure
```

① **Best-first search**

- Evaluation fn $f(n)$ → inversely proportional to estimate of desirability
  low $f(n)$ highly desirable
- Order the nodes in fringe in order of desirability
  ↳ priority queue (heap) is used here

⇒ special cases:
A* algorithm, greedy best first search

- BFS is best first search with $f(n) = \text{depth}(n)$
- Uniform cost search is best first search with $f(n) = \text{sum of edge cost from start state to } n$
  $h(n)$ → how far is the goal from n ; $g(n)$ ✓
  ↑ heuristic fn

② **Greedy best-first search** → algorithm that we can't trust completely

- evaluation fn: $f(n) = h(n)$
- Not complete can get stuck in loop (for tree search)
- Time complexity: $O(b^m)$ - but good heuristic can improve drastically
- Space complexity: $O(b^m)$
- Not optimal.

③ **A* search:**

$f(n) = g(n) + h(n)$

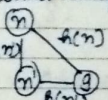
Admissible heuristic function:
→ heuristic function is admissible if it is optimistic (never overestimates the cost to reach the goal)

→ Admissible heuristic is less than, equal to optimal for minimisation problem and greater than or equal to optimal for maximization problem
↓ graph-search ⇒ check consistent

→ If $h(n)$ is admissible, A* using tree-search is optimal

Consistent Heuristics:

- $h(n)$ is consistent if for every node n and every successor n' due to legal action a
  triangular inequality is satisfied: $c(n,a,n')$
  $h(n) \leq c(n,a,n') + h(n')$



- Every consistent heuristic is admissible
→ If $h(n)$ is consistent, A* using graph search is optimal.

A* search:

- Complete unless there are infinitely many nodes with $f \leq f(A)$
- exponential time, keeps all nodes in memory
- Optimal depending upon search algo & heuristic property.

(x)

Iterative Deepening A* algorithm:

→ DFS till f increases till a certain bound.

(xi)

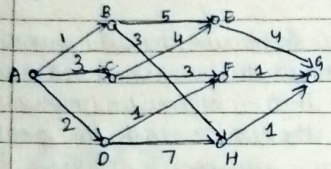
Depth first branch and bound: (DFS B&B)

2 mechanisms: branching & bounding → strength of this algo

→ branching – DFS with some preference on which node to visit first
↳ heuristic strategy

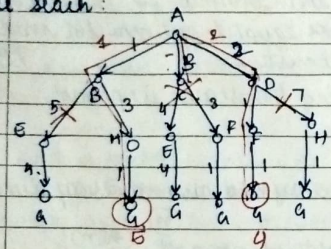
→ bounding - eliminate or terminate branch if it exceeds a certain bound

eg:



from A to G

true search:



We will maintain an upper bound (first we get 5) and a lower bound for expanding nodes

DFS B&B v/s IDA*

→ Both optimal but DFS B&B is systematic while IDA* is not
↳ if they finish

→ Memory is not exponential in both cases

→ IDA* never expands any node with f > optimal cost but DFS B&B expands suboptimal nodes.

→ IDA* better for infinite depth graph

DFS B&B is better if we have a better knowledge about upper bound
↳ if its easy to find suboptimal solution

(xii) ↳ suboptimal algo

Weighted A*: $f(n) = g(n) + w\,h(n)$ where $w > 1$, typically $w = 5$
↳ tried inadmissible heuristic

Admissible heuristics:

8 puzzle problem

$h_1(n)$ – no. of misplaced tiles

$h_2(n)$ – total Manhattan distance

$h_2(n) \geq h_1(n)$ for all n
$\Rightarrow h_2$ dominates h_1
↳ better for search

- A problem with fewer restrictions ⇒ relaxed problem.

The cost of the optimal solution in the relaxed problem is always going to be an admissible heuristic for the original problem
↳ for 8 puzzle problem, if you allow the neighbours to fly rather than just swaping with the neighbours but swaping only with empty space (that rule still exits) ⇒ optimal cost = ?

→ tile can move anywhere ⇒ $h_1(n)$ shortest soln

→ any adjacent square - a tile can move to ⇒ $h_2(n)$ shortest soln

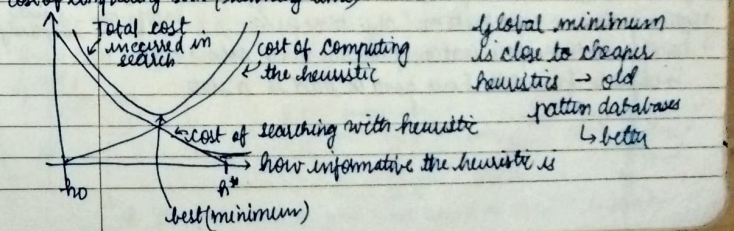
Hamiltonian cycle problem

State space - partial path, start state - empty

constraints: each node degree 2, visit all nodes, visit nodes exactly once

good heuristic for $a_1 \to a_2 \to \cdots \to a_k$

- length of shortest path from a_k to a_1
- length of MST of rest of the nodes (MST cost will be always less than the cost of path in hamiltonian cycle)



Linear Conflict Heuristic

↳ for 8 puzzle problem, if we use manhattan distance as the heuristic. 3-1 to 1-3 it tell 4 is the cost (2+2) but because of the interaction and overlaping, it'll take 6 min.

Pattern database Heuristic

- pattern database is a complete set of position with moves
- entire database computed by one backward BFS from the goal.
- all non pattern tiles are considered indistinguishable
- combining multiple database: eg red tile & blue tile (red tile - blue are considered indistinguishable and vice versa)
  - → when the heuristic give diffuent cost, the max is best heuristic; if sum, we will be double counting
  - ↳ Additive Pattern databases: we count pattern tiles movement ignoring non-pattern moves.
- → If no tile belongs to more than 1 pattern, add their heuristic value
- → manhattan distance - spl case when each pattern contains a singe tile.

Local search Algorithm:

→ solution is not the path but the state (each node will be a soln good or bad)
→ paths are irrelevant in some cases (N Queen)

Satisfaction v/s Optimization: Satisfaction is just satisfying all the constraints but optimization is satisfying but in optimal cost

→ Constraint satisfaction → optimization (minize the total number of constraints not satisfied).
→ Optimization → satisfaction (Objective fn $\geq$ / $\leq$ c) Optimization & satisfaction - same complexity
→ Local search - optimisation problem with soln: state
↳ Objective fn - tells how good or bad is a state

→ keeps track of a singe current state and only moves to neighbouring state
Advantage: very little memory, reasonable soln in large/infinite state space

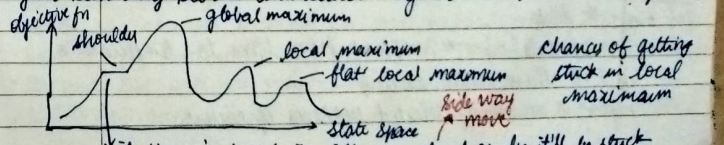
Pure optimization problem:
- All states have objective fn; goal - state with max/min objective fn
- → doesn't fit into path-cost / goal-state formulation
- ⇒ local search work well here.

N-Queens example: Satisfaction problem ⇒ optimization problem
- Objective function - no. of pairs of queens attack each other
  ↳ max: ${}^{N}C_2$ and min: 0 ⇒ Our goal is to minimize
  if we keep a constraint: only 1 queen in 1 column, ⇒ soln space N^N
  ↳ neighbourhood function ⇒ similar solution ~~have more than 1 queen~~
- ⇒ Systematic search v/s local search → search in soln space
  ↳ state is a partial configuration (ie 1st no queens)
- Neighbourhood function / Sucessor function - move a single queen to another square in the same column.

Trivial algorithm:
(i) Random sampling - generate a sample randomly
(ii) Random walk - pick a neighbour of the current state
} asymptotically complete

(13)
Hill climbing (Greedy Local search)

- Return a state that is a local maximum
- compute the objective function of all neighbours and move to max
  ↳ or first neighbour better than the state.
- Doesn't look ahead of the immediate neighbours
- You can only see the immediate neighbour and doesn't save past info



- Drawbacks: local maxima, plateaus, diagonal ridges
→ sideway move - keep a limit to avoid infinite loops

(14)
Tabu Search:
→ local search with a fixed length queue ("tabu list") add recent, drop oldest
→ prevent returning quickly to same state
→ as the size of tabu list grows, hill climbing will asymptotically become non-redundant (systematic ; 100 → practical tabu list size)

(15)
Enforced Hill climbing: escaping shoulders / local optima
perform BFS from local optima (middle ground b/n local search & systematic search)

(16)
Stochastic Hill climbing: with random walk, random restart sampling to make it asymptotically complete (local optima problem)
→ Random walk: at each step with probability p → greedy hill climbing and 1-p move to a random neighbour
→ Random restarts: ∞ different variations
↳ for each restart: run until termination v/s run fixed time
↳ Run a fixed no of restart or run indefinitely
→ both: at each step - greedy, random walk, random restart

(17)
Hill climbing with Simulated Annealing:
- pick randomly, δ is change in objective function
- if δ if positive, move to that state
- else move to that state with probability proportional to δ → use exponential fn
↳ worse move (large -ve δ) executed less often
- over time, make it less likely to make bad moves

Algorithm
for $t \leftarrow 1$ to ∞ do
$T \leftarrow$ schedule[t] → mapping from time to temperature
if $T=0$ then return current
next ← a randomly selected successor of current
$\Delta E \leftarrow$ value[next] - value[current]
if $\Delta E > 0$ then current ← next
else current ← next only with probability $e^{\Delta E/T}$
temperature: ↑ T, probability of bad move ↑, ↓ T, prob of bad moves
T↓ as algorithm runs ⇒ temperature schedule

(18)
Local Beam search: keep track of k states instead of 1.
- Select k randomly selected states, determine all their successors
- if successor → goal end else select k best successors & repeat
→ not same as k-random restart searches run parallely
→ problem: quite often all k state end up on same local hill
↳ soln: Stochastic beam search [k successors randomly, biased towards good ones]

(19) (20)
Genetic Algorithm: successor is generated by combining two parent states → State represented as a string over finite alphabet (0 & 1 or 2 5 7 4)
- start with k randomly generated states (population)
- Evaluation fn (fitness fn) - higher value better
↳ opposite to heuristic - for 8 queens no of non attacking pairs
- Production of next generation of state by simulated evolution
↳ Random selection
↳ Crossover
↳ Random mutation
population → fitness fn → natural selection → crossover (half & half from parents) → mutate at random (randomly change 1 node)
- Variant of stochastic beam search
- Negatives:
* large no of tunable parameters
* lack of good empirical studies
* no convincing evidence that better than hill climb with random restart

Adversarial search:
→ state, initial state, successor fn, terminal test, utility fn
- Successor fn: returns list of (move, state) pairs → legal move & resulting state
- terminal test: decides when the game is over
- Utility fn: gives a numeric value in states (-1 → loss, 0 → tie, +1 → win)

(2) Mini-max algorithm: (Similar to DFS but with no memory, both sides)
- move – by both players, ply – a half move (2 ply make a move)
- utility fn – fn applied to leaf nodes
- backed up value: of max-position → value of largest successor
  of min-position → value of smallest successor
- search down several levels; at bottom apply utility fn, back up all the way up to root node, select the node accordingly

*Multi agent system:
Game play – adversarial → if we win, others lose, • comp cooperative
- self interested agents – just optimising our utility fn (can be cooperative or adversarial according to situation)

→ complete if tree is finite
→ optimal for an optimal opponent and not against suboptimal opponents whose weakness could have been exploited.
→ Time complexity: $O(b^m)$, Space: $O(bm)$

(2) Alpha Beta pruning: to speed up minimax DFS search
- alpha – lower bound on value of max node ($V \geq \alpha$) initially $\alpha = -\infty$
- beta – upper bound on value of min node ($V \leq \beta$) $\beta = \infty$

→ when $\alpha >= \beta$ we can prune

*Bad case: Worst moves encountered first
↳ min part child in ascending & max descending
good case: Good moves ordered first
↳ min-descending, max-ascending
if we can order moves, better soln can be obtained.

*Time complexity: Normally → $O(b^m)$, with ideal α,β pruning: $O(b^{m/2})$
pruning → $T(m) = T(m-1) + (b-1)(T(m-2)) + c$

*Node Ordering: Iterative deepening search
use the evaluations of prev search for order, help return a move in given time.

(2) Minimax with cutoff
→ terminal – cutoff, utility – eval
→ Samuel's checkers – playing program
↳ uses linear evaluation function: $f(n) = w_1 f_1(n) + w_2 f_2(n) + \cdots + w_m f_m(n)$
↳ eg: $f = 6K + 4M + U$, K – king advantage, M – man advantage
U – undenied mobility advantage (no of moves that max where min can't go)
↳ learning mode: 2 players A & B; A adjusts its coefficient every move; B uses static utility fn; if A wins, its fn is given to B.

↳ Improvements:
- Horizon effect: how far you can look and what action you do accordingly
- Secondary search – search further down selected path to ensure this is the best move.
- Quiescence search – goes till quiescent state (stable), depth will be unknown

Refinements:
- Probabilistic cut – cut probabilistically based on shallow search & global depth level statistics
- Openings / endings – keep a catalog of best moves at especially start & end
- Singular extensions – find obviously good moves and try them at cutoff

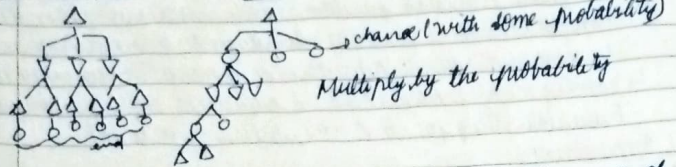
Deterministic games:
→ checkers – solved
→ chess – getting better
→ othello – computer is too good
→ go – till recently computer was too bad
↳ DeepMind's Alpha go

Alpha go (combination of Deep Neural Networks & Monte Carlo Tree Search)
Sample apart subtree and do intelligently & repeat

	deterministic	chance
perfect information	chess, checkers, go, othello	backgammon, monopoly
imperfect info	stratego	bridge, poker, scrabble

Game of chance (rolling die, picking card)
- 3 kind of nodes: min node, max node, chance node

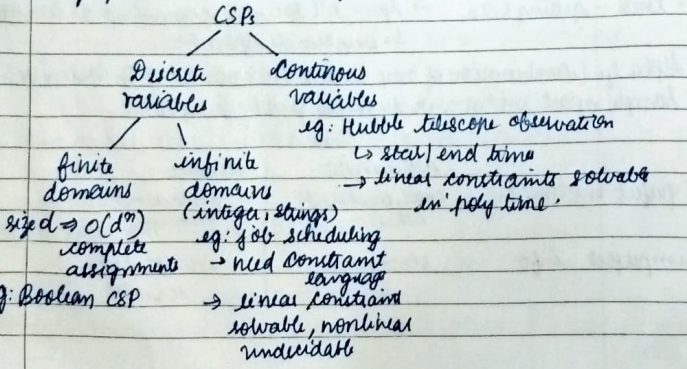
In minimax:



24

Expectiminimax algorithm: Time complexity $O(b^m n^m)$, n - no of chance outcomes (use pruning as the complexity is high)

Constraint satisfaction Problem: till now, it was search now it is representation of state (look inside state)
- Standard search problem: state - black box
- CSP: state - defined by variables x_i with values from Domain D_i
  goal state - set of constraints specifying allowable combinations of values for subsets of variables
- Binary CSP - each constraint relates at most two variables
  ↳ ~~Grid~~ map coloring ↳ convert any non binary to binary
  constraint graph:- nodes - variables, arc - constraints



Constraints:
→ Unary constraints - involves 1 variable eg: SA ≠ green
→ Binary constraints - pairs of variables eg: SA ≠ WA
→ higher order constraints - 3 or more, eg: cryptarithmetic
→ preferences (soft constraints) - constrained optimization problem

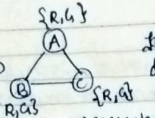
Search formulation for CSP:
- systematic search - initial state: not assigned any variables
  successor fn: assign a value to an unassigned variable without conflicts
  total leave: $n! d^n$ but d^n complete assignments only (if DFS)
  goal state: current assignment is complete
  ↳ here path is not important (we were using all permutations)
  → At a given nodes pick a local best variable
  ↳ backtracking search
- Who to choose - ~~the~~ MRV (Minimum remaining values)
  ↳ choose the variable with fewest legal values
  ↳ Tie breaking heuristic - degree heuristic
  ↳ choose the variable with most constraints on remaining variables
  ↳ least constraining value - one that rules out the fewest values in remaining variables

- Forward checking:
  - keep track of remaining legal values for unassigned variables
    terminate search when any variable has no legal value
  - doesn't provide early detection for all failures
  - constraint propagation: to repeatedly enforce constraints locally
- ↳ arc consistency - X → Y is consistent iff for every value x of X there is some ~~val~~ allowed y of Y
  ↳ for map coloring - if X loses a value, neighbours of X need to be rechecked

- Arc consistency ~~checks~~ detects failure earlier than forward checking

search - trying
inference - proving

- failure of arc consistency: (pairwise consistency)
- extension of arc consistency to general sets of k is called k consistency.
- detecting all failure is NP-hard as the CSP problem is NP-hard
- Most of the time k=2, some problem k=3 is the common practise



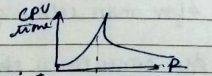
Here arc consistency can't detect any problem as it checks for pairs.

↳ versions of AC

* AC3 → $O(n^2d^3)$ can be $O(n^2d^2)$
AC-2 also ↳ AC-4

Cycle cut sets:
→ Independent subproblems - connected components of constraint graph
(↔ c variables out of n total ⇒ ~~d~~ $n/c - d^c$
→ loosely connected - one common node (branch on them first)
Tree structured CSPs: If constraint graph has no loops, CSP can be solved in $O(nd^2)$ time
Algorithm: convert the tree in topological sort. and go in reverse topological order and check arc consistency and assign them in forward path.
Nearly tree - structured CSPs:
cutset conditioning - instantiate a set of variables such that the remaining constraint graph is a tree.
cutset size - c ⇒ $O(d^c \cdot (n-c)d^2)$

Hill climbing search on CSPs
- soln space - full assignment of all the variables
- score - no. of constraints violated → minimize
- variable selection - randomly select any conflicted variable
- allow states with unsatisfied constraints, operators reassign variable values.
- value selection by min-conflicts heuristic: choose value that violates fewest constraints



R = no of constraints / no of variables

Logic in AI
Ontological commitment - what are the elements → propositions
Epistemological commitment - what do you know about them → True, false or IDK

Propositional Logic:
$a \rightarrow b \equiv \neg a \vee b$ (not a or b)

Q: If Asha is elected VP then Rajat is chosen as G-Sec and Bharati is chosen as Treasurer. Rajat is not chosen as G-Sec therefore Asha is not elected VP.

a: Asha is elected VP
b: Rajat chosen as G-Sec
c: Bharati is chosen as Treasurer

$F_1: (a \rightarrow (b \wedge c))$
$F_2: \neg b$
$G: \neg a$

check for $(F_1 \wedge F_2) \rightarrow G$

a	b	c	b∧c	a→(b∧c)	$F_1 \wedge F_2$	$(F_1 \wedge F_2) \rightarrow G$	
T	T	T	T	T	F	T	G is true
T	T	F	F	F	F	T	
T	F	T	F	F	F	T	
T	F	F	F	F	F	T	
F	T	T	T	T	F	T	
F	T	F	F	T	F	T	
F	F	T	F	T	T	T	
F	F	F	F	T	T	T	

Q: If Asha is elected VP then Rajat is chosen G-Sec or Bharati is chosen as Treasurer. Rajat is not chosen as G-Sec therefore if Asha is elected as VP then Bharati is chosen as treasurer.

$F_1: a \rightarrow (b \vee c)$
$F_2: \neg b$
$G: a \rightarrow c$
$F_1 \wedge F_2 \rightarrow G$

either a or b but not both
$F_1: a \rightarrow ((b \wedge \neg c) \vee (\neg b \wedge c)) : a \rightarrow b \oplus c$

- Interpretation - truth of a formula under assignment of boolean variables to true or false
- Valid - if it is true under all interpretation
  Non valid - if there is an interpretation for which it is false
- Satisfiable - if there is an interpretation for which it is true
  Unsatisfiable - if it is false under all interpretation.
- Boolean formula is decidable but NP-hard
  Truth table method, Tseitin method (faster)
  Data structures - Binary Decision Diagram (BDD)

Natural Deduction: (sound & complete)
1) Modus Ponens: (a→b), a therefore b
2) Modus Tollens: (a→b), ¬b therefore ¬a
3) Hypothetical Syllogism: (a→b), (b→c) therefore (a→c)
4) Disjunctive Syllogism: (a∨b), ¬a therefore b
5) Constructive Dilemma: (a→b) ∧ (c→d), (a∨c) therefore (b∨d)
6) Destructive Dilemma: (a→b) ∧ (c→d), (¬b∨¬d) therefore (¬a∨¬c)
7) Simplification: a∧b therefore a
8) Conjunction: a, b therefore a∧b
9) Addition: a therefore a∨b

Insufficiency of Propositional logic: all, some, whenever, whenever, every

Predicate Logic: first order predicate logic
↳ variables, some constants, predicate symbols and new connectors like ∃ (there exists), ∀ (for all)

goes(x,y) : x goes to y
- goes → predicate symbol
- x, y → variables

there exists → some
for all → all

Q: Wherever Mary goes, so does the lamb Mary goes to school. So the lamb goes to school.
Predicate: goes(x,y)
$F_1: \forall x\ (goes(Mary, x) \rightarrow goes(Lamb, x))$ $(F_1 \wedge F_2) \rightarrow G$
$F_2: goes(Mary, school)$ $G: goes(Lamb, school)$

Q: No contractor are dependable. Some engineers are contractors. Therefore some engineers are not dependable.
Predicates: contractor(x), dependable(x), engineer(x)
$F_1: \forall x\ (contractor(x) \rightarrow \neg dependable(x))$ / $\neg\exists x\ (contractor(x) \wedge dependable(x))$
$F_2: \exists x\ (engineer(x) \wedge contractor(x))$
$G: \exists x\ (engineer(x) \wedge \neg dependable(x))$ $(F_1 \wedge F_2) \rightarrow G$

Q: All dancers are graceful. Ayesha is a ~~student~~ dancer. Therefore Ayesha is a dancer. Therefore some students is graceful
Predicates: dancer(x), graceful(x), student(x)
$F_1: \forall x\ (dancer(x) \rightarrow graceful(x))$
$F_2: student(Ayesha)$ $F_3: dancer(Ayesha)$
$G: \exists x\ \{student(x) \rightarrow graceful(x)$ $(F_1 \wedge F_2 \wedge F_3) \rightarrow G$

Q: Every passenger is either in first class or second class. Each passenger is in second class if and only if the passenger is not wealthy. Some passengers are wealthy. Not all passengers are wealthy. Therefore some passengers are in second class.
Predicates: P(x), F(x), S(x), W(x)
$F_1: \forall x\ (P(x) \rightarrow (F(x) \vee S(x))$ $F_1': \forall x\ (P(x) \rightarrow (F(x) \oplus S(x)))$
$F_2: \forall x\ (P(x) \rightarrow (S(x) \rightarrow \neg W(x))) \wedge (\neg W(x) \rightarrow S(x)))$
$F_3: \exists x\ (P(x) \wedge W(x))$ $F_4: \neg(\forall x\ (P(x) \rightarrow W(x)))$
$F_4': \exists x\ (P(x) \wedge \neg W(x))$
$G: \exists x\ (P(x) \wedge S(x))$ $(F_1 \wedge F_2 \wedge F_3 \wedge F_4) \rightarrow G$

Q: Someone like everyone. Everyone likes someone. There is some one who everyone like. Everyone like everyone. If everyone likes everyone then someone like everyone. If there is a person whom everyone likes then that person likes himself. Predicate: likes(x,y) – x likes y

F_1: $\exists x(\forall y(likes(x,y))$

F_2: $\forall x(\exists y(likes(x,y))$ y depends on x

F_3: $\exists y(\forall x(likes(x,y)))$ y is independent of x

F_4: $\forall x \forall y$ like(x,y) / $\forall y \forall x$ likes(x,y)

F_5: $F_4 \rightarrow F_1$

F_6: $\exists xy(\forall x(likes(x,y)) \rightarrow likes(y,y)$

Laws of negation: Demorgan's law

$\neg(\forall x\, p(x)) \equiv \exists x(\neg p(x))$, $\forall x\, p(x) \equiv \bigwedge_x p(x)$, $\exists x\, p(x) \equiv \bigvee_x p(x)$

Q: If x is greater than y and y is greater than z then x is greater than z
The age of a person is greater than the age of his child
Therefore the age of a person is greater than the age of his grandchild
The sum of ages of two children are never more than the sum of ages of their parents

Function symbol: Age(x) Predicates: child(x,y) x is a child of y

$g(x,y)$ x is greater than y, parent(x,y)/child Sum(x,y) → fn Symbol

★ function symbol returns value while predicate returns T or F

F_1: $\forall x \forall y \forall z((g(x,y) \wedge g(y,z)) \rightarrow g(x,z))$

F_2: $\forall x \forall y$ (child$(x,y) \rightarrow g(Age(y), Age(x))$)

F_3: $\forall x \forall y g[Age(x), Age(child(y,x))]$

F_4: $\forall x \forall y\ g[Sum(Age(x), Age(y)), Sum(Age(child(x,x), child(y,y))]$

free and bound variables:

$\forall x(P(x,y))$ — x: bound, y: free

$\forall x\{P(x,y) \wedge \exists z\, q(x,y,z,w)\}$ — x: bound, y: free, z: bound, w: free

$\forall x\{P(x,y) \wedge \exists z \exists y\, q(x,y,z,w)\}$ — x: bound, y: free, z: bound, y: bound, w: free

- Variable can be quantified in first order predicate logic whereas symbols cannot $\exists p \forall x\{p(x)\}(x)$

Terminologies for predicate logic:

- Domain: D-specified for all interpretation
- constant symbols: M, N, O, P, ⋯
- variable symbols: x, y, z ⋯
- Function symbols: F(x), e(x)y
- Predicate symbols: p(x), q(x,y) ⋯
- connectors: ¬, ∧, ∨, →, ∃, ∀
- Term: variables, constant symbols are terms
  if $t_1, t_2, t_3, \cdots t_k$ is a term, $F(x_1, x_2, \cdots x_k)$ – fn symbol
  ⇒ $F(t_1, t_2, \cdots t_k)$ – term
  if $P(x_1, x_2, \cdots, x_k)$ – predicate symbol ⇒ $P(t_1, t_2, t_3, \cdots t_k)$ is
- Well-formed formula: proposition are wff
  Syntactically correct expression formed by combining symbols
- free & bound variables
- Interpretation – assign a domain set D, map constants, function, predicate suitably
- Validity ⇒ $(F_1 \wedge F_2) \rightarrow G$ is true under all interpretations
  Satisfiability $(F_1 \wedge F_2) \rightarrow G$ is true under atleast 1 interpretation

Limitations:

Undecidability / Halting

- Russell's Paradox – Barber shaves all those who don't shave themselves. There is a single barber in town. Who shaves barber?
  Checking validity of first order logic is undecidable but partially decidable (semi decidable) – Robinson's pictured of Resolution Refutation
- Higher order predicate logic can quantify symbols

Resolution Refutation for Propositional Logic:

To prove validity of $F = \{(F_1 \wedge F_2 \wedge \cdots \wedge F_n) \rightarrow G\}$ we shall attempt to prove that $\sim F = (F_1 \wedge F_2 \wedge \cdots \wedge F_n \wedge \sim G)$ is unsatisfiable

Steps:

1. Convert to clausal form / Conjunctive Normal form (CNF, Product of Sums)
2. Generate new clauses using the resolution rule
3. At the end, either False will be derived if the formula $\sim F$ is unsatisfiable implying F is valid.

Q: If Asha is elected VP then Rajat is chosen as G-Sec and Bharati is chosen as Treasurer. Rajat is not chosen as G-Sec. Therefore Asha is not elected VP.

a: Asha is elected VP, b: Rajat is chosen as G-Sec, c: Bharati is chosen as Treasurer

$F_1 : (a \rightarrow (b \wedge c)) = (\sim a \vee (b \wedge c)) = (\sim a \vee b) * \wedge (\sim a \vee c)$ (c_1, c_2)

$F_2 : \sim b$ (c_3), $G : \sim a$, $\sim G : a$ (c_4)

$\sim F = (\sim a \vee b) \wedge (\sim a \vee c) \wedge a \wedge \sim b$

$= \sim a \wedge a \wedge (\sim a \vee c)$

False

$\sim F = c_1 \wedge c_2 \wedge c_3 \wedge c_4$

Resolution Rule: Let $c_1 = a \vee b$ & $c_2 = \sim a \vee c$ then new clause $c_3 = b \vee c$ can be derived $[(c_1 \wedge (c_2) \rightarrow c_3]$

- To prove unsatisfiability use the Resolution Rule repeatedly to reach a situation where we have two contradictory clauses of the form $c_1 = a$ and $c_2 = \sim a$ from which False can be derived.

for propositional logic - the procedure terminates - sound & complete

Q: Rajesh either took the bus or came by cycle to class. If he came by cycle or walked to class he arrived late. Rajesh did not arrive late. Therefore he took the bus to class.

Bus: a, Cycle: b, Walk: c, ~~Late~~ late: d

$F_1 : (a \oplus b) \vee (a \wedge \sim b) = (\sim a \wedge \sim b) \vee (b \vee a) \wedge (\sim a \vee a) \wedge (\sim b \vee a)$ (c_1, c_2)

$F_2 : (b \vee c) \rightarrow d = \sim (b \vee c) \vee d = (\sim b \wedge \sim c) \vee d = (\sim b \vee d) \wedge (\sim c \vee d)$ (c_3, c_4)

$F_3 : \sim d$ (c_5)

$G : a$, $\sim G : \sim a$ (c_6)

$\sim F = F_1 \wedge F_2 \wedge F_3 \wedge \sim G$

$\sim F = (\sim a \vee \sim b) \wedge (b \vee a) \wedge (\sim b \vee d) \wedge (\sim c \vee d) \wedge \sim d \wedge \sim a$

c_1 c_2 c_3 c_4 c_5 c_6

$(c_2 \wedge c_6) \rightarrow c_7 : b$, ~~$(c_4 \wedge c_5) \rightarrow c_8 : \sim c$~~ $(c_3 \wedge c_7) \rightarrow c_8 : d$

$c_8 \wedge c_5 \Rightarrow$ False

Resolution Refutation for Predicate Logic:

create $F' = \sim F$ and check for unsatisfiability of F'

Steps: 1) Conversion to clausal form (CNF)

2) Applying Resolution Rule:

Concept of unification

Principle of Most General Unifier (mgu)

Repeated application of Resolution Rule using mgu

Conversion to clausal form in predicate logic:

1) Remove implications & other Boolean symbols converting to equivalent forms using $\sim, \vee, \wedge$
2) Move ($\sim$) inwards as close as possible
3) Standardize (Rename) variables to make them unambiguous
4) Remove existential Quantifiers by an appropriate new function/constant symbol taking into account the variables depend on the quantifier (Skolemization) ($\exists$)
5) Drop universal quantifiers ($\forall$)
6) Distribute $\vee$ over $\wedge$ and convert to CNF

Q: $\forall x$ (goes(Mary, x) $\rightarrow$ goes(Ramu, x)) : F_1

goes(Mary, School) : F_2

G: goes(Ramu, School)

F_1: $\forall x$ {goes(Mary, x)} $\vee$ goes(Ramu, x))

$\hookrightarrow c_1 : \neg$ goes(Mary, x) $\vee$ goes(Ramu, x) (5) universal

F_2: c_2 : goes(Mary, School)

$\sim G$: $c_3 : \neg$ goes(Ramu, School)

Substitution x as School, $(c_1 \wedge c_2) \rightarrow c_4$: goes(Ramu, School)

$c_4 \wedge c_3 \Rightarrow$ False

Q: $\forall x(\forall y(\text{student}(y) \rightarrow \text{like}(x,y)) \rightarrow (\exists z(\text{likes}(z,x)))$ to clausal form

$\forall x \{\neg(\forall y(\text{student}(y) \rightarrow \text{likes}(x,y))) \vee (\exists z(\text{likes}(z,x)))\}$

$\forall x \{\neg(\forall y(\neg(\text{student}(y) \vee \text{likes}(x,y)))) \vee (\exists z(\text{likes}(z,x)))\}$

$\forall x \{\exists y(\text{student}(y) \wedge \neg \text{likes}(x,y)) \vee (\exists z(\text{likes}(z,x)))\}$

Since $\exists y : F(x) = y$, $\exists z : G(x) = z$ Skolemization

$\forall x \{(\text{student}(F(x)) \wedge \neg \text{likes}(x, F(x))) \vee (\text{likes}(G(x), x))\}$

Drop universal quantifier: $(\text{student}(F(x)) \wedge \neg \text{likes}(x, F(x))) \vee \text{likes}(G(x), x)$

Skolemization

$\exists x \forall y\ P(x,y) \Rightarrow \forall y\ P(A,y) \Rightarrow P(A,y)$

$\forall x \exists y\ P(x,y) \Rightarrow \forall x\ P(x, F(x)) \Rightarrow P(x, F(x))$

Q: c_1: $\sim$studies$(x,y) \vee$ passes(x,y)

c_2: studies(Madan, z)

c_3: $\sim$passes(Chetan, Physics)

c_4: $\sim$passes(w, Mechanics)

c_1 & c_2 : In c_1 $x \rightarrow$ Madan, $y \rightarrow z \Rightarrow c_5$: passes(Madan, z)

$z \rightarrow$ Physics : c_6 : passes(Madan, Physics) c_5 is more general than c_6

c_4 & c_5 : $y \rightarrow$ Mechanics $\Rightarrow c_7$: $\neg$studies(x, Mechanics)

c_2 and c_7 $x \rightarrow$ Madan, $z \rightarrow$ Mechanics $\Rightarrow$ FALSE

Q: $\forall x(\text{contractor}(x) \rightarrow \sim\text{dependable}(x)) : F_1$

$\exists x(\text{engineer}(x) \wedge \text{contractor}(x)) : F_2$

G: $\exists x(\text{engineer}(x) \wedge \sim\text{dependable}(x))$

F_1 : $\forall x(\neg C(x) \vee \sim d(x)) \Rightarrow c_1$: $(\sim C(x) \vee \sim d(x))$

F_2 : $E(A) \wedge C(A) \Rightarrow c_2$: $E(A)$ c_3 : $C(A)$

$\sim G$: $\forall x(\sim e(x) \vee d(x)) \Rightarrow c_4 : (\sim e(x) \vee d(x))$

c_2 & c_4 : $x \rightarrow A \Rightarrow c_5$: $d(A)$

c_1 & c_5 : $x \rightarrow A \Rightarrow c_6$: $\sim C(A)$

c_3 & c_6 : False

Q: $\forall x(\text{dancer}(x) \rightarrow \text{graceful}(x)) : F_1$

F_2: student(Ayesha), F_3: dancer(Ayesha)

G: $\exists x(\text{student}(x) \wedge \text{graceful}(x))$

F_1 : $\forall x(\sim\text{dancer}(x) \vee \text{graceful}(x) \Rightarrow c_4 : (\sim d(x) \vee g(x))$

F_2 : $S(A) \Rightarrow c_2 : S(A)$, $F_3 : d(A) \Rightarrow c_3 : d(A)$

$\sim G$: $\forall x(\sim S(x) \vee \sim g(x)) \Rightarrow c_4 : (\sim S(x) \vee \sim g(x))$

c_1 & c_3 : $x \rightarrow A \Rightarrow c_5$: $g(A)$ $\Rightarrow c_6$ & c_2 : False

c_4 & c_5 : $x \rightarrow A \Rightarrow c_6$: $\sim S(A)$